

APPENDIX 15 – AGENT-BASED MODEL CODEBASE

Chapter 7 of this dissertation describes the design, construction, and analytical application of an agent-based model that computationally operationalizes the empirical findings of Stage 1 (Chapters 4 and 5). This appendix documents the computational system itself: its source code, data pipeline, analytical architecture, and the interactive observatory through which its capabilities are accessible. The complete codebase, comprising approximately 13,700 lines of code across 14 files, is available in a public repository. The interactive observatory is deployed as a web application.

Access

[Source code repository](https://github.com/dhruvdeepak-drp/Digital-Rights-Projects-ABM): [<https://github.com/dhruvdeepak-drp/Digital-Rights-Projects-ABM>]

[Interactive observatory](https://drp-abm.fly.dev): [<https://drp-abm.fly.dev>]

PIN: 1012

The source code repository contains the complete codebase and data files necessary to reproduce all of the model’s outputs. The interactive observatory provides a browser-based interface through which readers can explore the model’s analytical capabilities without installing software or running code. Access to the observatory requires a PIN, noted below the link. Running the code locally produces identical results to the deployed observatory.

Data Pipeline

The model reads empirical data from eight files produced during Stage 1 of the research:

- Phase 4B (condition positioning) provides quantitative condition scores for each of the 43 cases across all active condition types, establishing where each case falls on each condition range as described in Chapter 7.
- Phase 4D (outcome positioning) provides outcome scores across the four outcome groups.
- Phase 1A (master conditions and outcomes table) supplies the case equations, which the system parses into production rules: formalized triples of condition type, outcome group, and mechanism type that specify how particular conditions produce particular outcomes for each case.
- The Relational Density file provides partnership characteristics across four dimensions (partner diversity, relationship type range, network reach, and inter-case connectivity) as described in Chapter 7's data relations section.
- Phase 3A (quantitative matrix) and Phase 3B (qualitative matrix) supply the cross-impact assessments documenting how outcome categories relate to condition categories across the full case ecosystem.
- The Phase 3A condition and outcome mapping files provide the codebook linking category codes to their descriptions.

Upon loading, the system instantiates 43 agent objects, each carrying the full informational profile described in Chapter 7: internal and external condition scores, production rules, outcome scores, condition categories, outcome categories, and relational density characteristics. It then computes derived characteristics that are not stored in the data files but are calculated from the loaded data. Sovereignty capacity and terrain profiles are then computed. Condition diversity, outcome breadth, and the 17 recursive D-A-S pattern assignments are also computed during this initialization phase. Thirty-four automated tests validate that all 43 agents load with complete and internally consistent data.

Computational Architecture

The system is implemented in Python using the Flask web framework, with a single-page HTML front-end. It is organized into three functional layers: a data layer, a model layer, and an interface layer.

Data layer. The configuration file (`config.py`) defines the analytical vocabulary: the seven sovereignty dimensions, condition type and outcome category codebooks, the sovereignty mappings, mechanism type assignments, the 17 recursive D-A-S patterns, necessary condition constraints, and outcome co-occurrence relationships. The data loader (`data_loader.py`) reads the Excel files described above, instantiates agents, and computes derived characteristics. The agent class (`agent.py`) defines the data structure each agent carries. Together, these three files encode what Chapter 7 calls the ‘three-layer architecture’: condition scores positioning agents on sovereignty terrain, production rules specifying how conditions produce outcomes, and outcome scores constituting sovereignty capacity.

Model layer. The primary model engine (`primary_model.py`) holds all 43 agents and provides ecosystem-level access. The analytical engine (`analytical_engine.py`) implements the analytical operations described in Chapter 7: *condition sensitivity analysis*, which varies individual condition scores and measures the resulting changes in outcome scores using weights derived from both case-specific production rules and ecosystem-level cross-impact matrices; *counterfactual simulation*, which tests what happens when conditions are removed, maximized, or replaced with the profile of a different organizational type; *sovereignty pathway tracing*, which follows the chain from

conditions through production rules to outcomes to sovereignty dimensions for a given case; *hypothetical case construction*, which accepts user-specified condition configurations and generates predicted outcome and sovereignty profiles based on weighted similarity to empirical cases; and *reverse analysis*, which starts from a target outcome or sovereignty dimension and identifies which condition configurations would be required to achieve it. The relational engine (`relational_engine.py`) implements the second model described in Chapter 7: it holds all primary model variables constant and varies relational configurations (partnership diversity, relationship types, network reach, inter-case connectivity) to analyze how partnership ecosystems shape outcome and sovereignty production.

Interface layer. The Flask application (`app.py`) exposes the analytical engines through API endpoints. The front-end (`index.html`) provides nine interactive pages: a Dashboard summarizing the case ecosystem; a Case Explorer displaying individual agent profiles with condition configurations, production rule chains, outcome scores, and sovereignty mappings; a Comparison page enabling side-by-side case analysis; a Sensitivity Lab for varying condition scores and observing outcome effects; a Case Constructor for building hypothetical cases from user-specified conditions; a D-A-S Patterns page tracing the recursive design-affordance-sovereignty dynamics across conditions and outcomes; an Analytical Operations page exposing additional analytical capabilities; an Ecosystem page providing cross-case statistical views and scatter plot visualizations; and a Relational page implementing the second model's full analytical interface.

Production Rule Mechanics

The computational core of the primary model is the production rule system. Each case's equation (Appendix 4) was parsed into production rule rows: formalized relationships specifying that a particular condition type, operating through a particular mechanism, produces a particular outcome group. For example, a production rule might specify that Technology Practices (TP), operating through an enabling mechanism, produces Democratic Practices (DP) outcomes for a given case. The 43 cases collectively carry 184 such production rule rows.

When the analytical engine varies a condition score during sensitivity analysis, it traces which outcome groups are connected to that condition type through the case's production rules. Connected outcomes are then adjusted proportionally, with the magnitude of adjustment determined by two weight sources: the case-level production rule connections (whether a direct link exists) and the ecosystem-level cross-impact matrix (how strongly the condition type relates to the outcome group across all 43 cases). This dual-weight approach ensures that outcome changes reflect both the specific case's documented relationships and the broader empirical patterns. Outcomes not connected to the varied condition through either production rules or cross-impact evidence remain unchanged. The system evaluates these relationships configurationally rather than additively: specific combinations of conditions matter more than the total number present, consistent with the methodological principle described in Chapter 7.

For hypothetical case construction, the system identifies the five empirical cases most similar to the user-specified condition profile, measured by the sum of score

proximities across active condition types. Predicted outcomes are computed as weighted averages of those similar cases' outcome scores, with similarity serving as the weight. Predicted sovereignty profiles are computed analogously. This approach ensures that predictions remain grounded in empirical evidence: the model cannot predict outcomes that have no empirical precedent in the case ecosystem.

Sovereignty Computation

The system computes sovereignty through two distinct pathways, corresponding to Bridges B and D of the dissertation's theoretical architecture.

Sovereignty terrain (Bridge B) represents the sovereignty dimensions on which a case's conditions operate. Each of the eleven condition types is mapped to specific sovereignty dimensions based on the qualitative analysis described in Chapter 7. For each sovereignty dimension, a case's terrain score is the mean of its condition scores across all condition types that map to that dimension. Dimensions not engaged by any of the case's active conditions receive a terrain score of zero.

Sovereignty capacity (Bridge D) represents the sovereignty dimensions that a case's outcomes construct. Each of the 25 outcome categories is mapped to specific sovereignty dimensions based on what that outcome constructs: for example, DP7 (Legal and Rights-Based Protections) builds territorial and collective rights sovereignty. For each sovereignty dimension, a case's capacity score is the mean of the outcome scores across all outcome categories that map to that dimension, weighted by whether those categories are present in the case's outcome profile.

The gap between terrain and capacity is analytically significant. Where capacity exceeds terrain, there is emergent sovereignty: the case has constructed sovereignty capacity beyond what its immediate operating conditions would predict. Where terrain exceeds capacity, there is unrealized potential: the case operates on sovereignty dimensions that its outcomes have not yet fully engaged. The terrain-capacity gap analysis, available through both the analytical engine and the interactive observatory, reveals these patterns across the full case ecosystem and enables comparison across organizational types, geographic contexts, and condition configurations.

Relational Model Mechanics

The second model, implemented in the relational engine, addresses the question posed in Chapter 7: what happens when a case's partnership ecosystem changes while its internal conditions remain the same? The engine holds all condition scores, production rules, and outcome baselines constant. It then varies relational configurations across four density dimensions (partner diversity, relationship type range, network reach, and inter-case connectivity).

Relational sensitivity analysis works by identifying empirical cases with similar conditions but differing relational profiles. When a relational dimension is increased for a given case, the engine draws on the outcome profiles of empirically similar cases that have higher relational density to estimate how outcomes would shift. This approach preserves the empirical grounding: relational effects are not modeled abstractly but derived from documented differences across real cases. The engine also implements partner type effect analysis (what happens when specific partnership categories are added

or removed), agent-agent interaction analysis (for the seven cases with documented direct partnerships), and cluster analysis.

Codebase Reference

The table below lists all files in the repository with their size and purpose.

Table 1 Codebase description

File	Lines	Description
config.py	687	Analytical vocabulary and configuration. Defines the seven sovereignty dimensions, Bridge B condition-to-terrain mapping (CONDITION_SOVEREIGNTY_TERRAIN), Bridge D outcome-to-capacity mapping (OUTCOME_SOVEREIGNTY_MAP), mechanism type assignments, 17 recursive D-A-S pattern definitions, condition and outcome category codebooks, and canonical case name mapping.
data_loader.py	653	Data ingestion pipeline. Reads eight data files produced during Stage 1, instantiates 43 agents, and computes derived characteristics including sovereignty capacity profiles, sovereignty terrain profiles, condition diversity scores, and outcome breadth.
agent.py	173	Agent class definition. Each agent object carries internal and external condition scores, production rules (as condition-outcome-mechanism triples), outcome scores, sovereignty terrain and capacity profiles, relational density characteristics, and organizational metadata.
primary_model.py	373	Primary model engine. Provides ecosystem-level agent access, filtering by organizational type and other characteristics, and aggregation methods for cross-case analysis.
analytical_engine.py	1,782	Analytical operations on the primary model. Implements condition sensitivity, counterfactual simulation, sovereignty pathway tracing, hypothetical case construction, condition interaction analysis, terrain-capacity gap analysis, environment profiling, generative logic testing, and reverse analysis from target outcomes or sovereignty dimensions to required conditions.
relational_engine.py	1,661	Second model engine. Holds conditions constant and varies relational configurations across four density dimensions categories to analyze how partnership ecosystems shape outcome and sovereignty production. Implements density sensitivity, partner type effects, agent-agent interactions, cluster analysis, and relational moderation analysis.

app.py	1,399	Flask web application. Exposes all analytical capabilities through API endpoints and serves the interactive front-end. Handles agent serialization, request parsing, and response formatting.
index.html	5,991	Single-page interactive observatory. Contains all visualization logic, navigation, and user interaction across nine analytical pages; includes SVG scatter plots, and case exploration tools.
test_loading.py	221	Data loading validation suite (10 tests). Verifies all 43 agents load with complete condition, outcome, production rule, sovereignty, and relational data.
test_analytical.py	457	Primary model and analytical engine test suite (17 tests). Validates sensitivity analysis, counterfactual operations, sovereignty pathway tracing, and cross-case pattern detection.
test_relational.py	296	Relational engine test suite (7 tests). Validates density analysis, relational sensitivity, cross-impact calculations, and partnership pattern detection.
requirements.txt	3	Python package dependencies: Flask, openpyxl, gunicorn.
Dockerfile	12	Container specification for deployment.
fly.toml	22	Deployment configuration.

The repository contains all files necessary to reproduce the model’s outputs.

Running the system locally requires Python 3.10 or later and the three package dependencies listed in the requirements file (Flask for the web application, openpyxl for reading Excel files, and gunicorn for production deployment). The data files produced during Stage 1, which the model reads at startup, are included in the repository. A Dockerfile is provided for containerized deployment.

Relationship to Dissertation Chapters

Chapter 7 describes the data preparation that produced the files the model reads: the condition and outcome ranges, the scoring methodology, the case equations, the sovereignty mappings, and the relational density characterization. Every data structure in the system corresponds to a preparation step documented in that chapter. It also describes

the model's construction and analytical application, including the findings that emerged from the analytical operations implemented in the codebase. The interactive observatory provides direct access to the same analytical capabilities that generated the findings reported in Chapter 7, enabling readers to verify results, explore additional cases, and test alternative configurations. Appendix 4 contains the full set of 43 case equations from which the production rules are parsed. Appendix 11 documents the recursive logic analysis that informed the D-A-S pattern assignments computed during data loading. Appendix 12 contains the relational density scoring that the model reads from the Relational Density data file.